

Brazilian E-Commerce Delivery Performance Dashboard

Introduction

This project focuses on analyzing the delivery performance of a large Brazilian e-commerce marketplace using the publicly available **Olist E-Commerce Dataset** from Kaggle.

The goal of this project is to build an **interactive Tableau dashboard** that provide insights into customer behavior, order patterns, delivery performance, product demand, and seller activity.

The dataset contains **8 relational tables**, and we are using:

- **olist_orders_dataset** – order timestamps and delivery status and
- **olist_order_items_dataset** – item-level details including freight and shipping
- **olist_products_dataset** - product details

These tables were cleaned, prepared, and integrated to build a complete end-to-end Tableau project.

Olist Orders Dataset

The `olist_orders_dataset` table is the primary table in the Olist e-commerce database. It contains one row per order and includes key timestamps related to the order lifecycle, such as:

order_ID,

Customer_ID,

Order_Status,

order_purchase_timestamp - customer bought the product

order_approved_at - Payment approved.

order_delivered_carrier_date - date the order was shipped.

order_estimated_delivery_date - expected delivery date

order_delivered_customer_date -

If **not NaN**, it means delivery happened.

The Next step is to clean the orders data by importing the data into Jupyter Notebook using Python

The Query for Importing a CSV file is pasted as an Image Below :

```
import pandas as pd

orders_cleaned = pd.read_csv(r"C:\Users\User\Desktop\New folder (3)\olist_orders_dataset.csv", parse_dates=[
    'order_purchase_timestamp', 'order_delivered_customer_date', 'order_estimated_delivery_date' ])
orders_cleaned.head()
```

customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
104e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-02 11:07:15	2017-10-04 19:55:00	2017-10-10 21:25:13	2017-10-18
d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-26 03:24:27	2018-07-26 14:31:00	2018-08-07 15:27:45	2018-08-13
3443cd931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:55:23	2018-08-08 13:50:00	2018-08-17 18:06:29	2018-09-04
1dcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:45:59	2017-11-22 13:39:59	2017-12-02 00:28:42	2017-12-15
366dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:20:29	2018-02-14 19:46:34	2018-02-16 18:17:02	2018-02-26

The Orders_dataset was imported into Jupyter Notebook and parse_dates was used to Convert text dates into real datetime objects.

Next we would want to Keep only some columns in the orders_dataset and Remove the other columns the Query for that is pasted below :

```
orders_cleaned = orders_cleaned[['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp', 'order_delivered_customer_date', 'order_estimated_delivery_date']]
orders_cleaned.head()
```

order_id	customer_id	order_status	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date
1f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-10 21:25:13	2017-10-18
b2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-08-07 15:27:45	2018-08-13
0eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443cd931a367089	delivered	2018-08-08 08:38:49	2018-08-17 18:06:29	2018-09-04
d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcbec7375364d82	delivered	2017-11-18 19:28:06	2017-12-02 00:28:42	2017-12-15
1c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-16 18:17:02	2018-02-26

so we have eliminated order_delivered_carrier_date and order_approved_at and retained the remaining columns

Next we will count on Missing values of this dataset,

```
: orders_cleaned.isnull().sum()

: order_id                0
  customer_id             0
  order_status             0
  order_purchase_timestamp 0
  order_delivered_customer_date 2965
  order_estimated_delivery_date 0
  delivery_days           2965
  delay_days              2965
  is_late                  0
  dtype: int64
```

Now I see 2965 missing values in order_delivered_customer_date column but we see 0 missing column in order_purchase_timestamp , so the order_purchase_timestamp means definitely an order was placed by the the customer, so it has 0 Missing values and order_delivered_customer_date has missing values because

This is the date when the customer **actually received the order**.

This event **does NOT happen for every order**.

Why?

Because some orders were:

- ✗ Cancelled
- ✗ Returned
- ✗ Fraudulent orders
- ✗ Payment not approved
- ✗ Shipped but never delivered
- ✗ In transit but not delivered yet (rare but possible in dataset)

For these orders, the customer **never received** the product →

So order_delivered_customer_date is **NaN**.

I can also see that there are columns such as Delivery_days, Delays, Is_late which are created through python for easy calculation, so the query for the calculated columns is pasted below:

```
orders_cleaned['delivery_days'] = (orders_cleaned['order_delivered_customer_date'] - orders_cleaned['order_purchase_timestamp']).dt.days
orders_cleaned.head()
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-10 21:25:13	201
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-08-07 15:27:45	201
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-17 18:06:29	201
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbe7375364d82	delivered	2017-11-18 19:28:06	2017-12-02 00:28:42	201
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-16 18:17:02	201

```
orders_cleaned['delay_days'] = (orders_cleaned['order_delivered_customer_date'] - orders_cleaned['order_estimated_delivery_date']).dt.days
orders_cleaned.tail()
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date
99436	9c5dedf39a927c1b2549525ed64a053c	39bd1228ee81405908c3ca26f2dfe00	delivered	2017-03-09 09:54:05	2017-03-17 15:08:01	
99437	63943bddc261676b46f01ca7ac277bd8	1fca14ff2861355f6e5114306ff977a7	delivered	2018-02-06 12:58:58	2018-02-28 17:37:56	
99438	83c1379a015df1e13d02aae0204711ab	1aa71eb042121263aafbe80c1b562c9c	delivered	2017-08-27 14:46:43	2017-09-21 11:24:17	
99439	11c177c8e97725db2631073c19f07b62	b331b74b18dc79bcdf6532d51e1637c1	delivered	2018-01-08 21:28:27	2018-01-25 23:32:54	
99440	66dea50a8b16d9b4dee7af250b4be1a5	edb027a75a1449115f6b43211ae02a24	delivered	2018-03-08 20:57:30	2018-03-16 13:08:30	

```
[45]: orders_cleaned['is_late'] = (orders_cleaned['delay_days']).apply(lambda x: 1 if x > 0 else 0)
orders_cleaned.head()
```

```
[45]:
```

	order_id	customer_id	order_status	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-10-10 21:25:13	201
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-08-07 15:27:45	201
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-17 18:06:29	201
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbe7375364d82	delivered	2017-11-18 19:28:06	2017-12-02 00:28:42	201
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-16 18:17:02	201

So Now 3 New columns have been created and Now i am going to see the order_status of all these orders using value_counts

```
[49]: orders_cleaned['order_status'].value_counts()
```

```
[49]: order_status
      delivered    96478
      shipped      1107
      canceled     625
      unavailable  609
      invoiced     314
      processing   301
      created        5
      approved      2
      Name: count, dtype: int64
```

in the above image we can see how those 2965 missing is split into various types which is grouped in order_delivered_customer_date

Olist order Items Dataset

The `olist_orders_items_dataset` table is related to the primary table in the Olist e-commerce database. It contains one row per order item and includes key timestamps such as:

Order_Id,

order_item_ID,

Product_ID,

Shipping_limit_Date,- "Last date for seller to dispatch the item."

Seller_ID,

price,-The revenue/sales

Freight_value- Transportation and other extra charges

Now the Query for the imported data through the jupyter Notebook is shown below and `parse_dates` was used to Convert text dates into real datetime objects.

```
[39]: order_items_cleaned = pd.read_csv(r"C:\Users\user\Desktop\New folder (3)\olist_order_items_dataset.csv", parse_dates=['shipping_limit_date'])
order_items_cleaned.head()
```

```
[39]:
```

	order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.90	13.29
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-05-03 11:05:13	239.90	19.93
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-01-18 14:48:30	199.00	17.87
3	00024acbcd9a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-08-15 10:10:18	12.99	12.79
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-02-13 13:57:51	199.90	18.14

The second step is to look out for any missing values in the dataset, so the query is pasted below :

```
order_items_cleaned.isnull().sum()
```

```
order_id          0
order_item_id     0
product_id        0
seller_id         0
shipping_limit_date 0
price             0
freight_value     0
dtype: int64
```

This confirms there are no Missing values in the dataset, Next the third step is to check for duplicate values

The Next Steps is to check for any Duplicate values in the table , followed by the steps to check in any of the row if the price and freight_value is less than 0 or equal to 0

```
order_items_cleaned.duplicated().sum()
```

```
0
```

```
order_items_cleaned[order_items_cleaned['price'] <= 0]
```

```
Empty DataFrame
```

```
Columns: [order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value]
```

```
Index: []
```

```
filtered1 = order_items_cleaned[order_items_cleaned['freight_value'] < 0]
```

```
print(filtered1)
```

```
Empty DataFrame
```

```
Columns: [order_id, order_item_id, product_id, seller_id, shipping_limit_date, price, freight_value]
```

```
Index: []
```

so this confirms there are no duplicate values and the price and freight values is not less than 0

so the Next step is to add the price and freight value ,the query is pasted below:

```
order_items_cleaned['item_total'] = order_items_cleaned['price'] + order_items_cleaned['freight_value']  
order_items_cleaned.head()
```

order_id	order_item_id	product_id	seller_id	shipping_limit_date	price	freight_value	item_total
42fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	2017-09-19 09:45:35	58.90	13.29	72.19
77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	2017-05-03 11:05:13	239.90	19.93	259.83
9ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	2018-01-18 14:48:30	199.00	17.87	216.87
cbcdf0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	2018-08-15 10:10:18	12.99	12.79	25.78
y26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	2017-02-13 13:57:51	199.90	18.14	218.04

This Confirms a new column named as Item column is created and Next the order_items_cleaned is turned into a CSV file and imported into an Tableau worksheet and Connected to the orders dataset to create an data Model

Olist Products Dataset

The `olist_products_dataset` table is related to the primary table in the Olist e-commerce database. It includes product metadata such as:

product_ID,

product_category_names

product_name_length

product_description_length

product_photos_qty

product_weight_g
product_length_cm
product_height_cm
product_width_cm

First, the dataset was imported into Jupyter Notebook.

Then the following standard checks were performed:

Data types — using .info()

Duplicate records — using .duplicated().sum()

Preview — using .head()

```
[61]: product_dataset_cleaned = pd.read_csv(r"C:\Users\user\Desktop\New folder (3)\olist_products_dataset.csv")
      product_dataset_cleaned.head()
```

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria	40.0	287.0	1.0	225.0	1
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	44.0	276.0	1.0	1000.0	3
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.0	250.0	1.0	154.0	1
3	cef67bcfe19066a932b7673e239eb23d	bebes	27.0	261.0	1.0	371.0	2
4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas	37.0	402.0	4.0	625.0	2

```
[63]: product_dataset_cleaned.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  --
 0   product_id                32951 non-null object  
 1   product_category_name     32341 non-null object  
 2   product_name_lenght       32341 non-null float64 
 3   product_description_lenght 32341 non-null float64 
 4   product_photos_qty        32341 non-null float64 
 5   product_weight_g          32949 non-null float64 
 6   product_length_cm         32949 non-null float64 
 7   product_height_cm         32949 non-null float64 
 8   product_width_cm          32949 non-null float64
```

and Next step is checking for any Missing Values and the Query is pasted Below:


```
product_dataset_cleaned.isnull().sum()
```

```
product_id                0
product_category_name      610
product_name_length        610
product_description_lenght  610
product_photos_qty         610
product_weight_g           2
product_length_cm          2
product_height_cm          2
product_width_cm           2
dtype: int64
```

we could see 610 missing values in

- product_category_name
- product_name_length
- product_description_length
- product_photos_qty

2 missing values in:

- product_weight_g
- product_length_cm
- product_height_cm
- product_width_cm

The **610 missing values** all come from **the same products**, meaning:

These products were uploaded to Olist without any metadata.

The seller did not provide category, name length, description length, or number of photos.

This is very common in marketplace datasets:

- Sellers upload incomplete catalog data
- Old listings imported from other platforms
- Products created without full information

Because these fields do not affect sales calculations, we fill them as follows:

- `product_category_name` → `"unknown"`
- `product_name_length` → `0`
- `product_description_length` → `0`
- `product_photos_qty` → `0`

The Query To Fill Unknown and 0 through python In Jupyter Notebook is pasted Below:

```
[82]: product_dataset_cleaned.fillna({
      'product_category_name': 'unknown',
      'product_name_length': 0,
      'product_description_length': 0,
      'product_photos_qty': 0
    }, inplace=True)

[84]: numeric_cols = ['product_weight_g', 'product_length_cm', 'product_height_cm', 'product_width_cm']

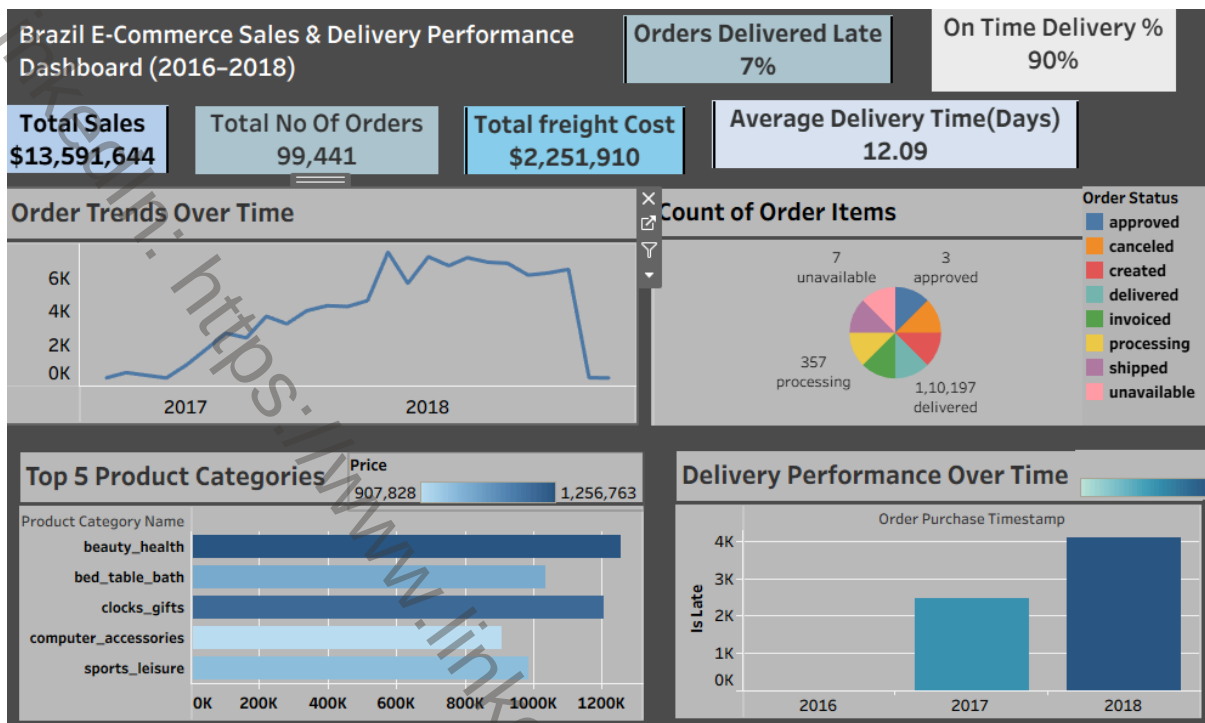
      for col in numeric_cols:
          product_dataset_cleaned.loc[:, col] =
              product_dataset_cleaned[col].fillna(product_dataset_cleaned[col].median())
```

Some Values are Filled with Median Because the dataset may have extremely small values and extremely large values basically they may have outliers, so it is better to fill with median(middle value) then mean(average value)

Next the `products_dataset_cleaned` is turned into a CSV file and imported into a Tableau worksheet and Connected to the orders dataset to create an data Model

Tableau Dashboard

This Tableau dashboard analyzes Brazil's e-commerce performance from 2016–2018. It highlights key metrics such as total sales, order volume, delivery performance, top product categories, and order status distribution. The dashboard helps identify trends, delays, and category-wise contribution to revenue.



So I would want To see what has this chart Revealed ,Lets Start with The KPI's

KPIs (Top of Dashboard)

Includes:

- Total Sales - \$13,591,644
- Total Number of Orders - 99,441
- Total Freight Cost - \$2,251,910
- Average Delivery Time (Days)-12.09 days
- Orders Delivered Late % - 7
- On-Time Delivery % - 90

This section Gives Overall Quick and valuable insights about the insights

Purpose:

Quickly summarizes overall business performance at a glance.

2. Order Trends Over Time (Line Chart)

Shows monthly order volume between 2016–2018.

Key Insight: Orders consistently increased until mid-2018.

3. Count of Order Items by Status (Pie Chart)

Breakdown of order processing stages (delivered, shipped, canceled, etc.).

Purpose: Understand operational workflow and bottlenecks.

4. Top 5 Product Categories (Bar Chart)

Highlights the product categories generating the highest revenue.

Key Insight: Beauty & Health and Clocks & Gifts dominate sales.

5. Delivery Performance Over Time (Bar Chart)

Shows how late deliveries changed from 2016 to 2018.

Purpose: Evaluate logistics performance and improvement areas.

Key Insights

- Sales Grew Steadily by the year end of 2016 and reached the Top of sales at the Mid of 2017 and Dropped by the end of 2018 due to data limitations or operational issues.
- Top categories show consumers prefer beauty, health, and home goods.
- 2016 had zero late deliveries, 2017 saw a moderate increase, and 2018 experienced the highest number of late deliveries. This indicates a decline in delivery performance over the years."

Tools Used

- Tableau

- pandas (data cleaning)